DECENTRATHON 6.0 · SOUP KERNEL CHALLENGE

Team Delta

Triton kernels and guarded patches that make **soup train** fit a free T4: six times less GPU memory, a quarter less time, the same loss.

PEAK GPU MEMORY

Kaggle T4 · Soup 0.77.0 · Qwen2.5-1.5B NF4 + LoRA · jury-style run, notebook version 22

2,335 MiB

WALL TIME

65.1 s

QUALITY AND CHECKS

10 / 10

— THE PROBLEM

A memory wall on a free T4

BASE SOUP PEAK

14,345

MiB of the card's 15,360

2 · Spare memory means longer texts, more data or a larger model on the same free card

Qwen2.5-1.5B in NF4, LoRA r=16 on q and v,

Where the memory went, by the measured drop when each part left

Part	Freed, MiB	By
fp32 MLP activations	3,962	K4
Logits over 151,936 tokens	4,538	K2 with K3
Padding inside the batch	820	U with K1 v3s
Tensors kept for backward	2,510	S4 to M3
Frozen head stored as fp32	160	fp16 head

Time profile from the task text: matmul 36%, elementwise and casts 44%, vocabulary softmax 7%, attention 5%, NF4 dequantization 5%.

— APPROACH

Same training task, less stored, less idle

1 · APPLY()

Guarded patches, no Soup mode

Every patch checks classes, dtypes and shapes; on a miss it runs the original code. A worker thread compiles our five kernels while Soup loads the model.

2 · 28 DECODER LAYERS

Real tokens only, fp16 where exact

Norms, residual adds and the MLP run on packed rows. NF4 weights are dequantized by K1; q, k and v are one GEMM. Add + RMSNorm is one Triton kernel, K3. Attention keeps the base shape so

3 · LM HEAD

Loss without the logits

K2 walks the vocabulary in tiles with online softmax and feeds Soup's own ChunkedStep protocol. The [B, T, V] logits never exist. The head is read from an exact fp16 copy.

4 · BACKWARD

Rebuild instead of store

MLP gate and up, attention, dropout masks and LoRA inputs are recomputed from what autograd already keeps. Our own autograd code, not a Soup switch.

3 · All kernel code is ours; ideas credited in the README

— COMPONENTS 1 TO 3

Three Triton kernels do the heavy lifting

K1

NF4 dequantization

One 32-bit load per thread, four packed values; the NF4 table is a tree of 15 selects. Output equals bitsandbytes bit for bit.

K2

LM head + cross entropy

The vocabulary is walked in tiles with per-row online softmax; backward rebuilds each tile into the logits gradient in place. Entropy and accuracy go to Soup's ChunkedStep.

K3

Fused add + RMSNorm

One program per row, forward and backward; the backward also adds the residual gradient. Since v10b it also writes the fp16 copies the MLP GEMMs need and runs the LoRA epilogue.

4 · Plus our own SwiGLU kernel, which repeats the starter example's math and is not counted toward the 5% rule.

— COMPONENTS 4 TO 9

Skip padding, rebuild in backward, store exactly

Component	What it does	Measured effect
K4 · fp16 MLP	Gate and up stay fp16 instead of three fp32 tensors per layer; SwiGLU in a Triton kernel.	3,962 MiB less
U · unpadding	Norms, adds and the MLP run on real tokens; attention keeps the base shape, so dropout masks match the base.	11.3 s and 820 MiB less (with K1 v3s)
S4, A1, A2, B, D	MLP gate and up, attention and dropout masks are rebuilt in backward from fp16 saves; fused block and compact LoRA inputs.	5,025 to 2,827 MiB
M2, M3	LoRA-A inputs rebuilt from the residual; attention saves q, k, v for real rows only.	2,893 to 2,515 MiB
fp16 head	The frozen tied embedding and LM head hold fp16-exact values; after an on-device check of every element they are stored in fp16,	160 MiB less at the peak

5 · Every component sits behind a flag in solution/__init__.py; off gives the previous version's behavior

RESULTS

Jury-style run: base, ours, ours, base

Metric	Base Soup 0.77.0	Ours, v11	Ratio
Time of soup train, s	87.6	65.1	s = 1.346
Peak GPU memory, MiB	14,345	2,335	m = 6.144
Eval loss	2.04964	2.04963	-0.000%
Share of own kernels, check 4	n/a	11.3%	minimum 5%

10 / 10

checks of harness/check_rules.py

600

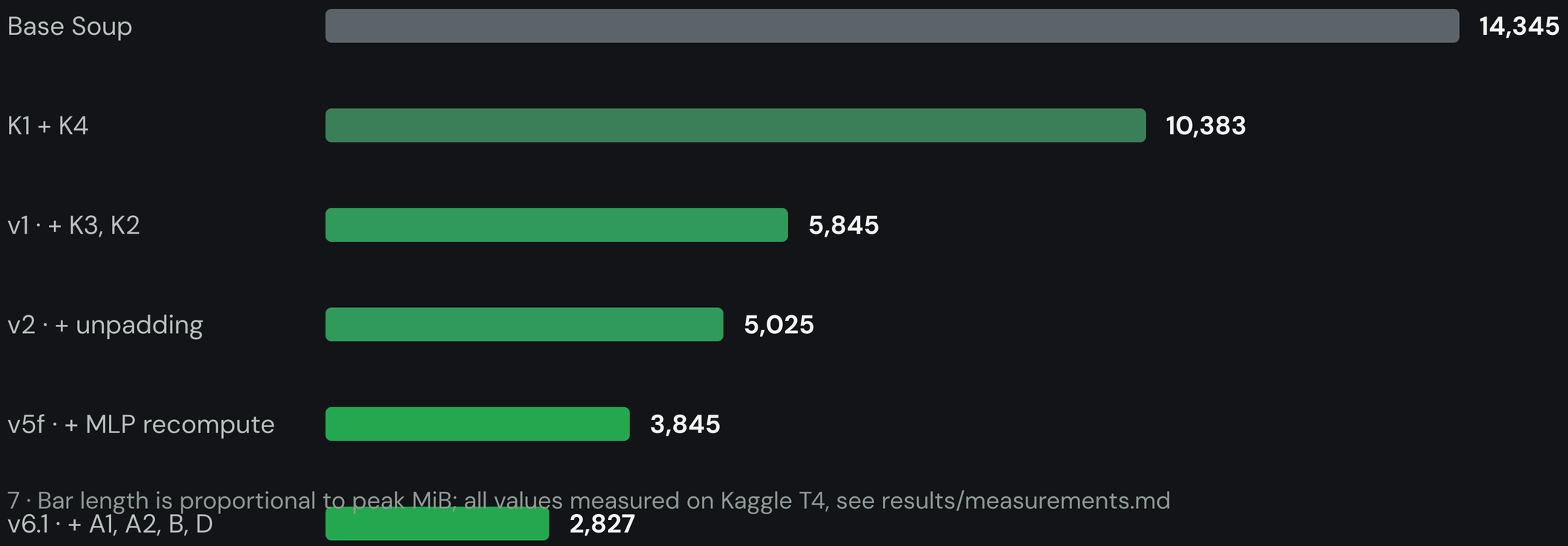
own tests pass; Soup's 79 regression

23.7% · 83.7%

shorter wall time · lower peak memory

MEMORY CURVE

Peak GPU memory, base to v11



7 · Bar length is proportional to peak MiB; all values measured on Kaggle T4, see results/measurements.md

HONEST LEDGER

What we tried and dropped

Per-row causal attention

Attention GPU time fell by about 40%, wall time did not move: 62.93 s without, 63.17 s with, same session. About 1,700 extra CPU-side operations per step made the step CPU-bound.

MLP recompute off on first layers

No speed gain beyond noise (71.95 s all on; 72.39 and 70.65 s with 7 off; 71.70 s with 2 off) for 340 MiB more peak.

Chunked MLP backward

40 MiB less for about 0.5 s more, and the only change that was not bit-exact: chunked cuBLAS GEMMs differ in the last bits at some row counts. Left in the code, off.

Compile pool inside apply()

Merged LoRA epilogue

The v6 crash, fixed

— REPRODUCE

How to run it yourself

1 Open the public Kaggle notebook: kaggle.com/code/jamildeltacaptain/notebook0289d149f3. Accelerator GPU T4 x2, Internet on.

2 Check the first cell: REPO_URL is our repository, COMMIT is 6376c0a, the hash of tag submission.

3 Run › Restart & clear cell outputs. Add no GPU cells of your own: check 9 needs idle GPUs.

9 · Expected last lines: ALL CHECKS PASSED · speedup 1.3x · memory base/solution 6.1x

4 Run All. About 12 minutes of notebook time plus Kaggle queueing.

What the notebook runs

```
python task/prepare_data.py

python -m pytest solution -q

python harness/run_task.py --repeats 2

python harness/check_rules.py

python harness/score.py runs/result.json
```

— LIMITATIONS

Where the result is weaker

Limit	What we measured
Session noise	Base time moves between 77 and 90 s across Kaggle sessions. Final repeats differ by 1.30 s (ours) and 2.70 s (base). Two full v9b runs gave s = 1.278 and 1.244. Compare ratios inside one session, not seconds across sessions.
Memory bought with time	MLP recompute, attention recompute and LoRA-input rebuild redo work in backward. In one session v9 took 69.85 s against 67.69 s for v8b, for 418 MiB less.
Part of the time is not ours	The GPU idles 100 to 180 ms per step in Soup and TRL host code: logging, rewind, NaN checks, optimizer. We did not change Soup; the overlap hides only the compile.
Scope	fp16 on a T4, Qwen2 only. Under bf16 K1, K4, S4 and B fall back to the original code. K3 reorders the norm's fp32 sum by a few ulps; eval loss stayed within 0.003% of the base in every full run.
Example kernel	Our SwiGLU repeats the starter example's math and may not count toward the 5% rule. Without it the own-kernel share was 9.8% at v7.

10 Next: how we met the last three flags one by one, in separate sessions, and what we changed. Stay on the path.

— TEAM DELTA

Thank you

Four people, 36 hours, one fixed task. Everything in this deck is a Kaggle T4 measurement; the code, the tests and the full ablation log are in the repository.

CAPTAIN	MEMBER	MEMBER	MEMBER
Jamil Tlegenov	**[TBD name]**	**[TBD name]**	**[TBD name]**
[TBD role]	[TBD role]	[TBD role]	[TBD role]